软 件 工 程

实验三：软件实现与构建

王睿阳 241880548

2026 年 11 月

项目：外卖点餐与配送系统（校园食光）

目录

一、实现概述 **1**

二、UML 到代码的转换 **2**

1 类图 → 领域模型 2

2 与设计的差异及回改 ..... None

3 代码规模 ..... None

4 大模型辅助开发过程 ..... None

二、编译与运行结果 ..... **None**

1 运行方式 7

2 主链路运行 8

3 自动化验证 8

三、Git 远程仓库管理 ..... **None**

一、实现概述

本项目延续实验一的需求分析与实验二的设计建模，选题不变：单校区外卖点餐与配送系统「校园食光」，覆盖顾客、商家、配送人员、平台管理员四类角色。实验三的任务是把实验二的 8 张 UML 图落地为可运行的软件。

整体架构按实验二组件图同进程落地，共四层：

层	代码位置	职责
客户端层	campus_eats/ui	四角色界面 + 登录，只经网关调服务
网关	services/gateway.py	会话管理、角色鉴权（REQ-001）
应用服务层	campus_eats/services	网关 + 八个领域服务 + 调度器 + 时钟
基础设施层	campus_eats/repositories	SQLite 仓储，对应组件图的数据库

二、UML 到代码的转换

1 类图 → 领域模型

类图六个包逐一映射为 models/ 下的模块：用户与权限（account.py）、店铺与菜品（shop_dish.py）、购物车与订单（cart_order.py）、支付与退款（payment_refund.py）、配送（delivery.py）、审计与通知（audit.py），枚举集中在 enums.py。

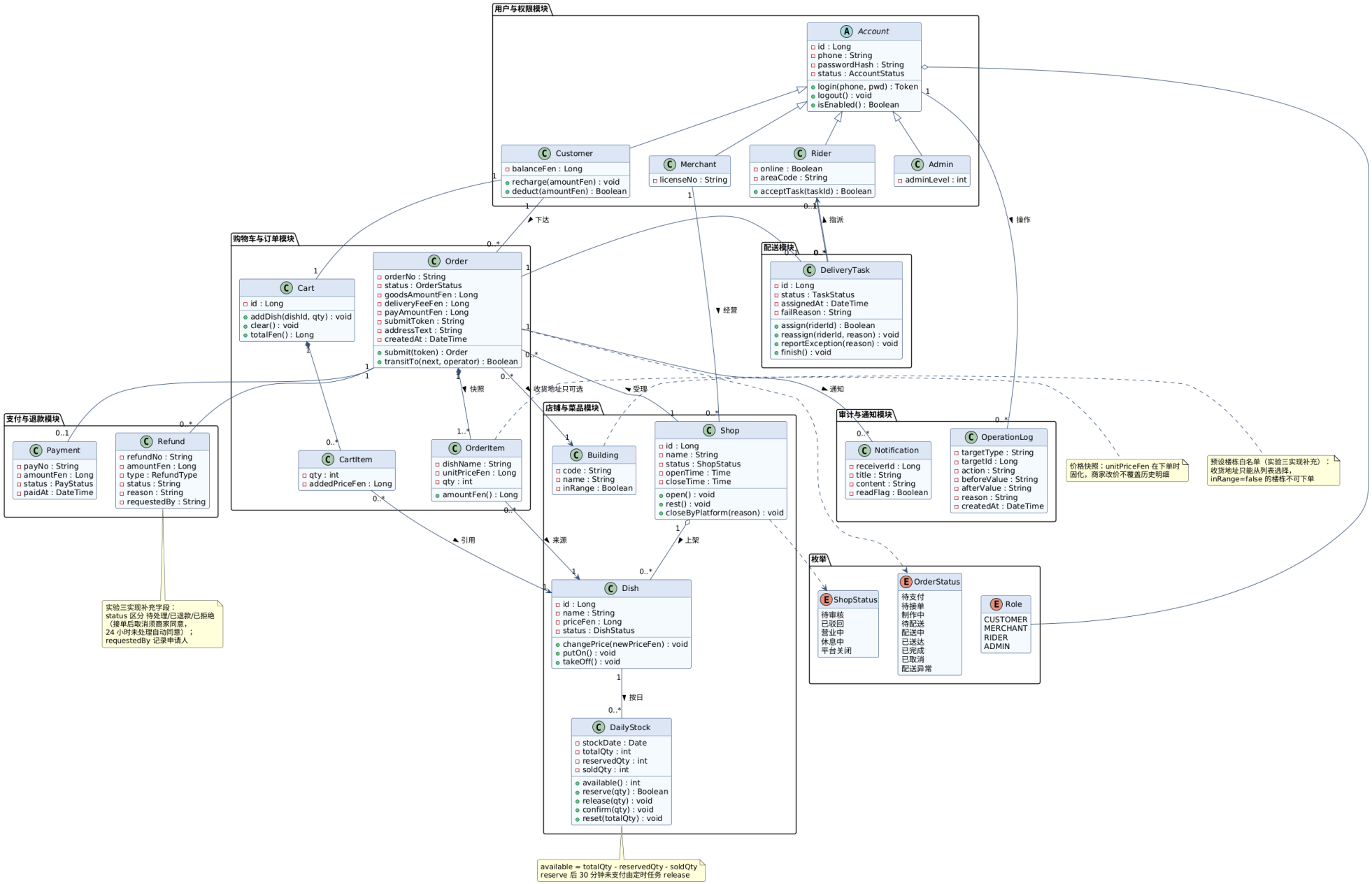
对于独立UML图中每个箭头，转换成状态机状态的改变，而组件图本身翻译为目录结构和服务类。状态转换就是方法调用，对于会相互依赖的服务，我采用类似dsh的总线思想，每个服务类是一个插件，互相不干扰，独立构造，最后用app.py黏合到一起。同时各个uml图之间的联系与状态转变也被放到总线管控，这样可以保证模块的独立性和流程的可读性较好；

2 与设计的差异及回改

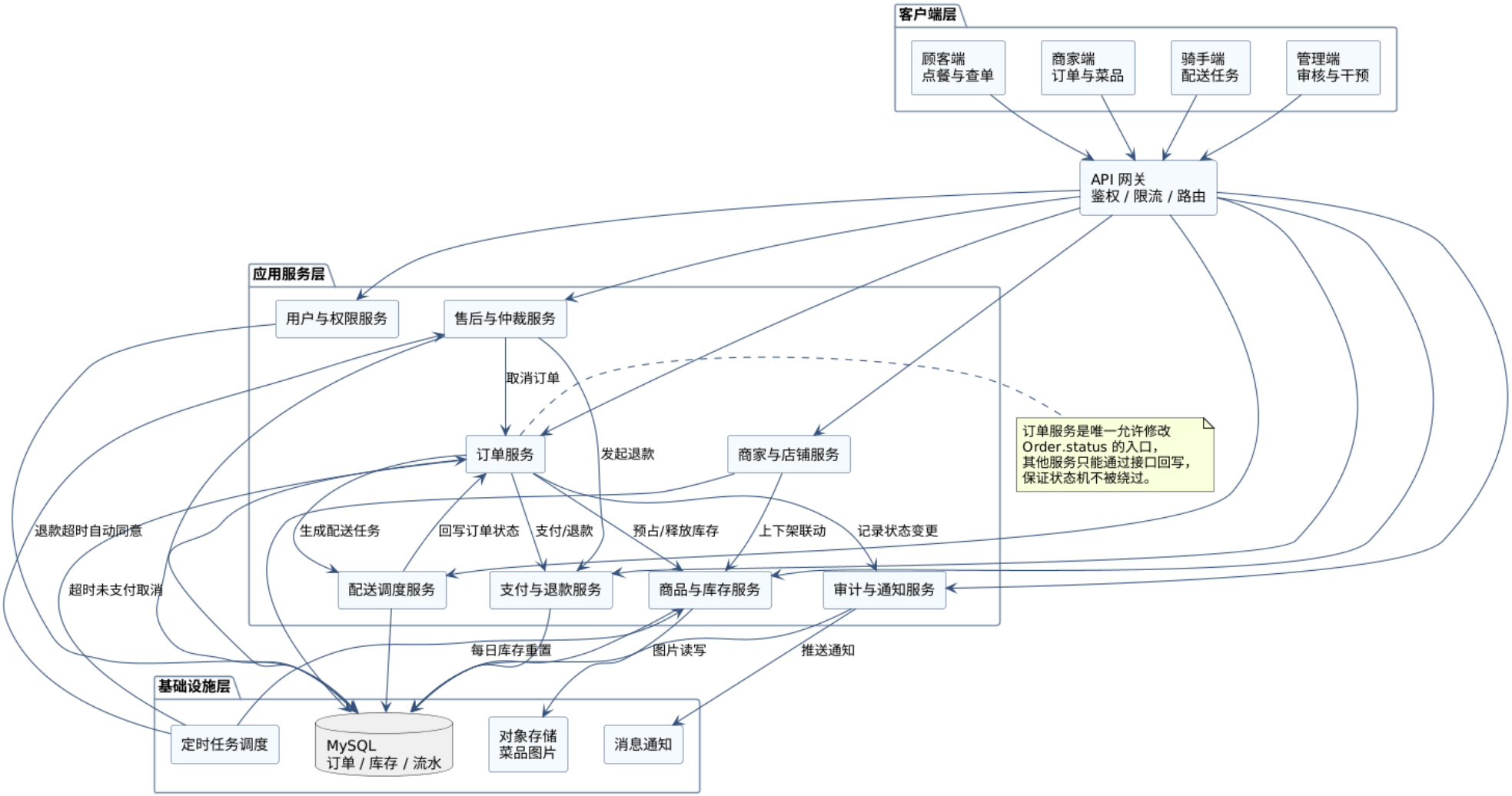
实现与实验二设计有五处出入，按实验二报告第 6 章的约定"以实现为准回改设计"，类图已更新并重新渲染（提交 7949c53）：

1. OrderStatus 补「配送异常」挂起态——状态图里有、类图枚举漏了
2. ShopStatus 补「待审核 / 已驳回」前置态——REQ-002 的入驻审核流程需要
3. 新增 Building 预设楼栋类——REQ-008 要求地址只能从白名单选择
4. Refund 补 status / requestedBy 字段——"24 小时未处理自动同意"需要退款状态机
5. login / acceptTask 两个方法从实体移入服务

3 代码规模


用户与权限模块
Account
id : Long
phone : String
passwordHash : String
status : AccountStatus
login(phone, pwd) : Token
logout() : void
isEnabled() : Boolean
Customer
balanceFen : Long
recharge(amountFen) : void
deduct(amountFen) : Boolean
Merchant
licenseNo : String
Rider
online : Boolean
areaCode : String
acceptTask(taskId) : Boolean
Admin
adminLevel : int
下达
指派
操作
经营
购物车与订单模块
Cart
id : Long
addDish(dishId, qty) : void
clear() : void
totalFen() : Long
Order
orderNo : String
status : OrderStatus
goodsAmountFen : Long
deliveryFeeFen : Long
payAmountFen : Long
submitToken : String
addressText : String
createdAt : DateTime
submit(token) : Order
transitTo(next, operator) : Boolean
配送模块
DeliveryTask
id : Long
status : TaskStatus
assignedAt : DateTime
failReason : String
assign(riderId) : Boolean
reassign(riderId, reason) : void
reportException(reason) : void
finish() : void
快照
收货地址只可选
受理
通知
支付与退款模块
Payment
payNo : String
amountFen : Long
status : PayStatus
paidAt : DateTime
Refund
refundNo : String
amountFen : Long
type : RefundType
status : String
reason : String
requestedBy : String
CartItem
qty : int
addedPriceFen : Long
OrderItem
dishName : String
unitPriceFen : Long
qty : int
amountFen() : Long
店铺与菜品模块
Building
code : String
name : String
inRange : Boolean
Shop
id : Long
name : String
status : ShopStatus
openTime : Time
closeTime : Time
open() : void
rest() : void
closeByPlatform(reason) : void
审计与通知模块
Notification
receiverId : Long
title : String
content : String
readFlag : Boolean
OperationLog
targetType : String
targetId : Long
action : String
beforeValue : String
afterValue : String
reason : String
createdAt : DateTime
价格快照：unitPriceFen 在下单时固化，商家改价不覆盖历史明细
预设楼栋白名单（实验三实现补充）：收货地址只能从列表选择，inRange=false 的楼栋不可下单
引用
来源
上架
枚举
ShopStatus
待审核
已驳回
营业中
休息中
平台关闭
OrderStatus
待支付
待接单
制作中
待配送
配送中
已送达
已完成
已取消
配送异常
Role
CUSTOMER
MERCHANT
RIDER
ADMIN
Dish
id : Long
name : String
priceFen : Long
status : DishStatus
changePrice(newPriceFen) : void
putOn() : void
takeOff() : void
按日
DailyStock
stockDate : Date
totalQty : int
reservedQty : int
soldQty : int
available() : int
reserve(qty) : Boolean
release(qty) : void
confirm(qty) : void
reset(totalQty) : void
available = totalQty - reservedQty - soldQty
reserve 后 30 分钟未支付由定时任务 release
实验三实现补充字段：
status 区分 待处理/已退款/已拒绝（接单后取消须商家同意，24 小时未处理自动同意）；
requestedBy 记录申请人

UML类图


客户端层
顾客端
点餐与查单
商家端
订单与菜品
骑手端
配送任务
管理端
审核与干预
API 网关
鉴权 / 限流 / 路由
应用服务层
用户与权限服务
售后与仲裁服务
取消订单
订单服务
发起退款
商家与店铺服务
订单服务是唯一允许修改
Order.status 的入口，
其他服务只能通过接口回写，
保证状态机不被绕过。
退款超时自动同意
生成配送任务
回写订单状态
支付/退款
预占/释放库存
上下架联动
记录状态变更
超时未支付取消
配送调度服务
支付与退款服务
商品与库存服务
审计与通知服务
每日库存重置
图片读写
推送通知
基础设施层
定时任务调度
MySQL
订单 / 库存 / 流水
对象存储
菜品图片
消息通知

UML组件图

规模统计（build/count_lines.py，口径：代码行 = 含非注释 token 的物理行去重）：

目录	文件数	总行数	代码行
campus_eats	44	5806	4430
tests	7	868	717
scripts	2	388	309
合计	53	7062	5456

满足"大于 1500 行"的要求，其中不含任何 GUI 设计工具生成物——界面全部为手写 QSS 与布局代码。

4 大模型辅助开发过程

1 分工与工作流

本次实验使用 ZCode（CLI Agent，模型 GLM）。分工与实验二一脉相承：**我负责总体架构与核心思路控制**——我先确定软件架构与每个服务的大致实现思路，具体代码生成由模型处理；**模型承担实现主体**——从领域模型、仓储、八个服务到 PySide6 界面与测试的代码产出；**自动检查链兜底**——pylint 静态检查、离屏界面冒烟、32 项端到端验收、63 项 pytest，每层产物先过检查再进下一层。

2 主要使用记录

记录一：实现计划与选型。 把题目、27 条需求、8 张 UML 源码交给模型要求给出计划并列出需要人工拍板的决策点，同时告诉模型这个项目的总体架构组成，独立构建每个类的服务；

记录二：提供服务类实现思路 对于每个服务类的实现，都在模型开始工作前提供大致思路，避免模型使用不受掌控的逻辑构建代码，加强代码的可控性；

记录三：设计类CI流程 为了进一步保证软件质量，告诉ai要设计类似CI的自动化检测与单元测试脚本，这样可以控制ai的幻觉。如每层代码写完、提交之前，固定跑一组自动检查，pylint 打分等；

3 生成内容不符合预期时的处理

十次值得记录的修复里，问题被三种手段分别抓住，边界很清晰：

静态检查抓得住的：例如money.py 漏 import 自己引用的异常类（pylint E0602）、字典与括号风格告警。这类问题在"写完一层立即 lint"的流程下马上就能暴露，叫ai再改即可。

跑起来才暴露的：例如配送任务的状态迁移方法 _transit 只做合法性校验、从不赋值 status——类型正确、逻辑缺失，静态检查完全无感，但仍然被我要求的benchmark拦下来，告诉ai错因即可。

只有读语义才发现的：这种是测试都发现不了的问题，最难处理；如令牌单元测试：想测令牌重放，却生成了新令牌，等于没测。它跑绿了，我后续复查代码才发现逻辑不对，是我提供的spec没包含才导致这样的impl，只有说清楚ai才不会幻觉自己实现；

同时每次修复我都同时把"这类错怎么被自动发现"补进检查链，让同类问题不出现第二次。

4 对辅助开发的思考

以下大多都是jyy老师给我提供的心得：

- 只有通过**抽象**，把复杂的东西**分层**，**分别考虑其中的复杂度**，才好维护
- **agent loop**：**反复 review**
- **多个 LLM 互相 review**
- **BFS 试错**（用广度展开试错，而不是一条路走到黑）
- **CI/CD 自动化检验与提交**
- 核心原则：**所有开发者都是不可信的**，**机器永远是对的**
- 用这套东西来**约束 agent**（不靠自觉，靠可执行的检查）

二、编译与运行结果

1 运行方式

环境：Python 3.10.11，依赖 PySide6 / pytest / pylint（见 requirements.txt）。

`powershell

cd lab3

python -m venv .venv

.venv\Scripts\pip install -r requirements.txt

.venv\Scripts\python -m campus_eats --demo

`

--demo 为演示模式，时间 60 倍加速。首次运行自动建库并写入种子数据：2 家店铺、6 个菜品、3 名骑手、7 个预设楼栋（含 1 个配送范围外楼栋用于 REQ-008 验收）。演示账号：顾客 13800000001、商家 13900000001、骑手 13700000001、管理员 13600000001，密码均为 123456。

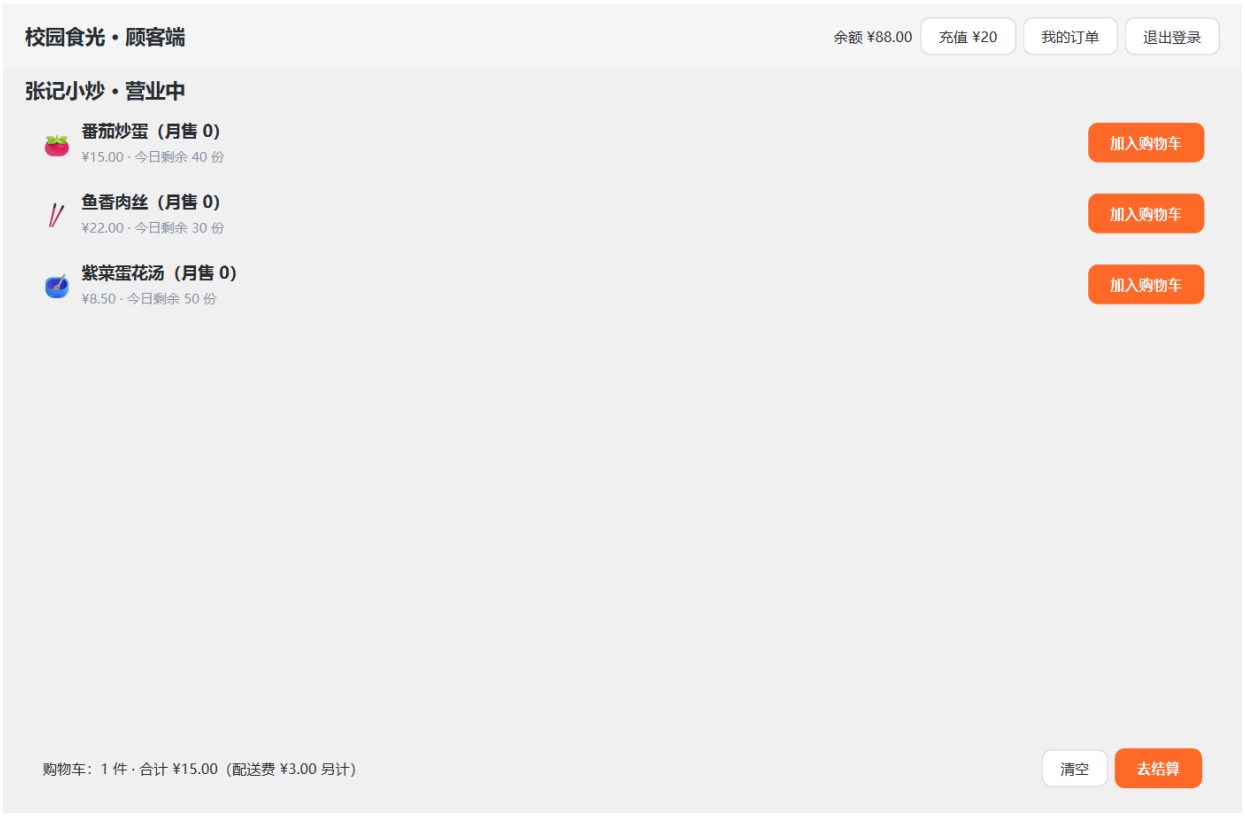
2 主链路运行



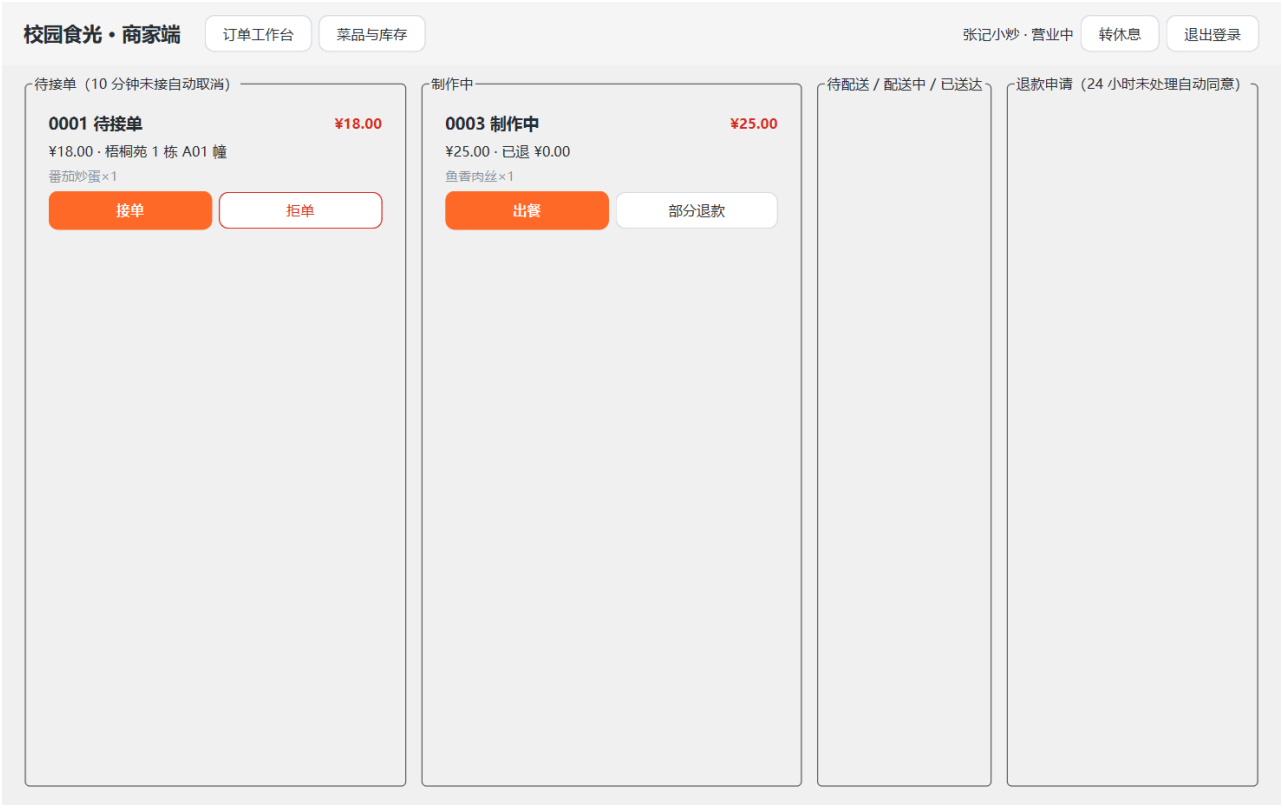
登录页：登录后按角色进入对应端



确认订单：楼栋下拉限选（范围外禁选），提交即预占库存并锁价



顾客端店铺详情：菜品行显示剩余份数，底部购物车条实时合计



商家端订单工作台：待接单 / 制作中 / 履约中 / 退款申请分区



管理端派单与干预：待派送队列、账号封禁与操作记录

主链路在 demo 模式下可完整走通：浏览加购 → 提交订单（预占库存并锁价）→ 余额支付 → 商家接单出餐 → 管理员按片区派单 → 骑手接受并送达 → 顾客确认收货。超时规则可现场观察：待支付订单放置约 30 秒被自动取消并回补库存；支付后商家不接单，约 10 秒自动取消并全额退款。

3 自动化验证

验证项	命令	结果
单元与集成测试	python -m pytest	63 passed
代码风格	python -m pylint campus_eats	10.00/10
端到端验收	python scripts/e2e_check.py	32 项全过
并发不超卖（REQ-023）	200 线程抢库存 10	恰好 10 单，库存归零不为负

并发用例用 200 个线程同时提交，断言成功订单恰好等于库存份数——这是实验一 REQ-023 的验收标准在原型上的直接落地。

三、Git 远程仓库管理

仓库延续实验一、二：github.com/marcus-37/lab1-takeout，main 分支，本次推送区间 ba7e9b2 → 1029c55。实验三放在仓库 lab3/ 子目录，实验一要求"每人建立自己的公开仓库"，二三未另行要求，因此不另建仓库。

提交按层划分，每个提交对应一次可验证的完成：

提交	内容
d8bd71f	脚手架：依赖清单、包骨架、pylint 配置、AI 记录
4906424	领域模型层：类图六包映射、状态机迁移表
c8fba63	SQLite 仓储层（含子类 role 缺陷修复）
a7a35c4	应用服务层：网关、八服务、调度器，e2e 32 项通过
b2f5278	PySide6 界面：登录 + 四角色端
fe3ae0d	pytest 套件 63 项 + 两个统计缺陷修复
7949c53	按实现回改实验二类图并重渲染
1029c55	报告管线、统计、打包脚本

两点说明：一是实验一固定的 lab1-ai-draft / lab1-final 标签全程未动；二是类图回改单独提交且落在 lab2 目录——实验二报告约定"以实现为准回改设计"，这个提交就是那次回改的记录，与实现提交分开，历史可追溯。